CCL Lab Manual

Q1] Create three IAM users: user-1, user-2, and user-3, and assign them to appropriate IAM groups. Each group should have specific permissions attached according to the organizational policy.

User	Group	Permission
user-1	S3-Support	Read only access to Amazon S3
user-2	EC2-Support	Read only access to Amazon EC2
User-3	EC2-admin	View,start and stop amazon ec2 instances

Tasks:

1. Create the required IAM groups: S3-Support, EC2-Support, and EC2-Admin.
2. Attach appropriate managed policies to each group based on the permissions specified.
3. Create IAM users: user-1, user-2, and user-3.
4. Implement Multi-Factor Authentication (MFA) for all users and enforce it using an IAM policy.
5. Assign each user to the corresponding group.
6. Verify that permissions are correctly applied by testing access levels.|

Mapped to : Lab 1

Steps:

PART 1 — LOGIN TO AWS

Step 1: Open AWS Console

Open: https://aws.amazon.com/console/

Step 2: Login

Enter:

- AWS account email
- Password

Click: Sign In

PART 2 — OPEN IAM SERVICE

Step 3: Search IAM

At top search bar type: IAM

Click: IAM

PART 3 — CREATE GROUPS

Create Group 1 → S3-Support

Step 4: Left side menu → User groups

Step 5:

Click: Create group

Step 6:
Group name:
S3-Support
Step 7:
Under permissions policies search:
AmazonS3ReadOnlyAccess
Tick checkbox.
Step 8:
Click:
Create group
Create Group 2 → EC2-Support
Step 9:
Again click: Create group
Step 10:
Group name: EC2-Support
Step 11:
Search policy: AmazonEC2ReadOnlyAccess
Tick checkbox.
Step 12:
Click: Create group

Create Group 3 → EC2-Admin
Step 13:
Click: Create group
Step 14:
Group name: EC2-Admin
Step 15:
Search: AmazonEC2FullAccess
Tick checkbox.
Step 16:
Click: Create group

PART 4 — CREATE IAM USERS
Create user-1
Step 17:
Left menu → Users
Step 18:
Click: Create user
Step 19:
Username: user-1
Step 20:
Tick: Provide user access to AWS Management Console
Step 21: Choose: I want to create an IAM user
Step 22: Password type:

Custom password
Step 23:
Password: User@123
Step 24:
Click: Next

PART 5 — ADD USER TO GROUP
Step 25:
Choose: Add user to group
Step 26:
Tick: S3-Support
Step 27:
Click: Create user

Create user-2
Step 28:
Click: Create user
Step 29:
Username: user-2
Step 30: Enable console access
Step 31:
Password: User@123
Step 32:
Add to group: EC2-Support
Step 33:
Click: Create user

Create user-3
Step 34:
Click: Create user
Step 35:
Username: user-3
Step 36: Enable console access
Step 37:
Password: User@123
Step 38:
Add to group: EC2-Admin
Step 39:
Click: Create user

PART 6 — ENABLE MFA
Enable MFA for user-1
Step 40:
Open:Users

Step 41:
Click: user-1
Step 42:
Open tab: Security credentials
Step 43:
Under MFA click: Assign MFA device
Step 44:
Select: Authenticator app
Step 45:
Open:Google Authenticator on mobile.
Step 46: Scan QR code.
Step 47: Enter first OTP.
Step 48: Enter a second OTP.
Step 49:
Click: Assign MFA
Step 50:
Repeat MFA steps for:

- user-2
- user-3

PART 7 — TEST ACCESS
Test user-1
Step 51:
Logout admin account.
Top right profile → Sign Out
Step 52:
Login using IAM URL: https://account-id.signin.aws.amazon.com/console
Step 53:
Enter: Username: user-1
Password: User@123
Step 54: Enter MFA OTP.

Test S3 Access
Step 55:
Search: S3
Step 56:
Open S3.
Expected: User can view S3 buckets

Test EC2 Access
Step 57:
Search: EC2
Expected: Access Denied

Test user-2
Step 58: Logout.
Login as: user-2
Password: User@123
Enter MFA OTP.

Test EC2
Step 59: Open EC2.
Expected: Can view EC2

Try Stop Instance
Step 60:
Select instance →Instance State → Stop Instance
Expected: Access Denied

Test S3
Step 61:
Search: S3
Expected: Access Denied

Test user-3
Step 62: Logout.
Login as: user-3
Password: User@123
Enter MFA OTP.

Test EC2 Full Access
Step 63: Open EC2.
Expected: Full access available
Step 64: Select instance.
Step 65:
Click: Instance State
Step 66:
Click: Stop Instance
Expected: Instance stops successfully

Q2] Using Amazon Web Services, demonstrate storage provisioning, backup, and data recovery using Amazon EBS with Amazon EC2.
Task

1. Create an Amazon EBS volume.
2. Attach the volume to an Amazon EC2 instance.
3. Configure and mount the file system on the instance.
4. Store sample data on the mounted volume and verify it.
5. Create an EBS snapshot for backup.

6. Restore data by creating a new volume from the snapshot.
7. Attach the restored volume to the EC2 instance.
8. Mount and verify that the recovered data is intact.

Mapped to : Lab 4

Steps:
PART 1 — OPEN EC2
Step 1: Login to AWS Console.
Step 2:
Search: EC2
Click EC2.

PART 2 — LAUNCH EC2 INSTANCE
Step 3:
Click: Launch Instance
Step 4:
Name: MyServer
Step 5:
Choose AMI: Amazon Linux
Step 6:
Instance type: t3.micro
Step 7:
Under Key Pair:
Click: Create new key pair
Step 8:
Key pair name: mykey
Step 9:
Key pair type: RSA
Step 10:
Private key format: .pem
Step 11:
Click: Create key pair
Key downloads automatically.
Step 12:
Under Network Settings click: Edit
Step 13:
Allow: SSH traffic from Anywhere
Step 14:
Click: Launch Instance

PART 3 — CREATE EBS VOLUME
Step 15:
Left menu → Volumes
Step 16:
Click: Create Volume

Step 17: Volume type: gp2
Step 18:
Size: 8 GB
Step 19:
Availability Zone: Choose SAME AZ as EC2 instance.
Example: ap-south-1a
Step 20:
Click: Create Volume

PART 4 — ATTACH VOLUME TO EC2
Step 21: Select the created volume.
Step 22:
Click: Actions
Step 23:
Click: Attach Volume
Step 24: Select EC2 instance:
MyServer
Step 25:
Device name:/dev/xvdf
Step 26:
Click: Attach Volume

PART 5 — CONNECT TO EC2
Windows Users
Step 27: Open: PowerShell
Step 28: Copy Public IP from EC2 instance.
Example: 13.xx.xx.xx
Step 29:
Open Powershell and run: ssh -i mykey.pem ec2-user@public-ip
Example: ssh -i mykey.pem ec2-user@13.xx.xx.xx
For Mac: run first this to give permission:chmod 400 mykey.pem
Step 30:
Type: yes
Press Enter.
Connected successfully.

PART 6 — CHECK NEW DISK
Step 31:
Run: lsblk
You will see: /dev/xvdf

PART 7 — FORMAT EBS VOLUME
Step 32:
Run: sudo mkfs -t ext4 /dev/xvdf

Wait until formatting completes.

PART 8 — CREATE DIRECTORY
Step 33:
Run: sudo mkdir /data

PART 9 — MOUNT VOLUME
Step 34:
Run: sudo mount /dev/xvdf /data
Step 35:
Verify mount: df -h
You should see: /data

PART 10 — STORE FILE INSIDE VOLUME
Step 36:
Run: sudo sh -c 'echo "AWS Practical Data" > /data/file.txt'
Step 37:
Verify file: cat /data/file.txt
Expected output: AWS Practical Data

PART 11 — CREATE SNAPSHOT
Step 38: Go back to AWS Console.
Step 39:
Open: Volumes
Step 40: Select attached volume.
Step 41:
Click: Actions
Step 42:
Click: Create Snapshot
Step 43:
Description: Backup Snapshot
Step 44:
Click: Create Snapshot
Snapshot creation starts.

PART 12 — CREATE NEW VOLUME FROM SNAPSHOT
Step 45:
Left menu → Snapshots
Step 46: Select created snapshot.
Step 47:
Click: Actions
Step 48:
Click: Create Volume from Snapshot
Step 49: Select SAME Availability Zone as EC2.

Example:ap-south-1a
Step 50:
Click:
Create Volume

PART 13 — ATTACH RESTORED VOLUME
Step 51:
Open: Volumes
Step 52: Select the newly created volume.
Step 53:
Click: Actions
Step 54:
Click: Attach Volume
Step 55:
Select instance: MyServer
Step 56:
Device name: /dev/xvdg
Step 57:
Click: Attach Volume

PART 14 — MOUNT RESTORED VOLUME
Step 58:
Go back to the terminal.
Step 59:
Check disks:lsblk
You should see:/dev/xvdg
Step 60:
Create folder: sudo mkdir /restore
Step 61:
Mount restored volume: sudo mount /dev/xvdg /restore

PART 15 — VERIFY RECOVERED DATA
Step 62:
Run: cat /restore/file.txt
Expected output: AWS Practical Data

Q3] To Store and retrieve files using AWS S3
Task

1. Open Amazon S3 from the AWS Console.
2. Create a new S3 bucket with default settings.
3. Upload files such as images and text documents.
4. Configure public access permissions for uploaded files.
5. Access and verify the uploaded files using S3 URLs.

Mapped to: Experiment 7 (refer deep)

Steps:
PART 1 — LOGIN TO AWS
Step 1:
Open AWS Console.
Login using AWS Academy account.

PART 2 — OPEN S3 SERVICE
Step 2:
In AWS search bar type: S3
Click: S3

PART 3 — CREATE S3 BUCKET
Step 3:
Click: Create bucket
Step 4:
Bucket name:
Example: mybucket12345
Important:

- Bucket name must be unique
- If error comes, add extra numbers

Example:
mybucket1234589
Step 5:
Region: Keep default.
Example: Asia Pacific (Mumbai) ap-south-1
Step 6:
Under: Object Ownership
Keep default: ACLs enabled
Step 7:
Under: Block Public Access settings
Untick: Block all public access
Step 8:
Tick acknowledgement checkbox: I acknowledge that the current settings might result in this bucket and the objects within becoming public
Step 9: Keep all remaining settings default.
Step 10:
Click: Create bucket
Bucket created successfully.

PART 4 — OPEN BUCKET
Step 11:
Click bucket name: mybucket12345

PART 5 — UPLOAD FILE

Step 12:
Click: Upload
Step 13:
Click: Add files
Step 14:
Select any file from the computer.
Example:

- image
- pdf
- text file
- HTML file

Step 15:
Click: Open
Step 16:
Click:Upload
Wait until upload completes.

PART 6 — MAKE FILE PUBLIC
Step 17: Click on the uploaded file.
Step 18:
Click: Actions or object actions(on top right corner)
Step 19:
Click: Make public using ACL(click on the action ,dropdown will open choose the last option)
Step 20:
Click: Make public
Now the file becomes public.

PART 7 — RETRIEVE FILE USING URL
Step 21: Click uploaded file name.
Step 22:
Scroll down.
Copy: Object URL
Example: https://mybucket12345.s3.ap-south-1.amazonaws.com/file.jpg
Step 23:
Open a new browser tab.
Paste URL.
RESULT
Expected: Uploaded file opens successfully

Q4] Using Amazon Web Services, create and configure a relational database instance using Amazon RDS.
Task

1. Amazon RDS from the AWS Console.
2. Create a new database using Standard Create.

3. Select MySQL or PostgreSQL.
4. Configure the DB instance class
5. Create administrator username and password.
6. Configure storage within Free Tier limits.
7. Enable public access for remote database connectivity.
8. Enable automatic backups for the database instance.
9. Connect to the database using a database client (e.g., MySQL Workbench/psql).
10. Verify connectivity by creating a sample table and inserting records.

Mapped to : Lab 5

Steps:

PART 1 — LOGIN TO AWS

Step 1:

Open AWS Console.

PART 2 — OPEN RDS SERVICE

Step 2:

Search: RDS

Click: RDS

PART 3 — CREATE DATABASE

Step 3:

Click:Create database (Full configuration)

Step 4:

Under:

Choose a database creation method

Select: Full configuration

Step 5:

Engine type:

Select: MySQL

Step 6:

Engine Version:

Keep default.

Step 7:

Under Templates select: Free tier

PART 4 — DATABASE SETTINGS

Step 8:

DB instance identifier: mydb

Step 9:

Master username: admin

Step 10:

Master password: admin123

Step 11:

Confirm password: admin123

PART 5 — INSTANCE CONFIGURATION

Step 12:

DB instance class:

Keep default:

db.t3.micro

Step 13:

Storage:

Keep default settings.

PART 6 — CONNECTIVITY SETTINGS

Step 14:

Under Connectivity: Keep default VPC.

Step 15:

Public access:

Select: Yes

Step 16:

VPC Security Group:

Choose: Create new

Step 17:

Security group name: rds-sg

Step 18:

Availability Zone:

Keep default.

PART 7 — ADDITIONAL CONFIGURATION

Step 19:

Initial database name: studentdb

Step 20:

Keep all other settings default.

BACKUP CONFIGURATION

Step 1:

Open:

Additional Configuration

Step 2:

Under Backup:

Ensure:Enable automated backups is selected.

Step 3:

Backup retention period: 1 days

Step 4:

Keep the remaining settings default.

Step 21:

Click: Create database

PART 8 — WAIT FOR DATABASE

Step 22:

Wait 5–10 minutes.
Status changes from:
Creating
to:
Available

PART 9 — MODIFY SECURITY GROUP

Important: Without this step MySQL Workbench cannot connect.

Step 23:

Click database: mydb

Step 24:

Open: Connectivity & security , then click on endpoints tab

Step 25:

Click security group link: rds-sg

Step 26:

Click: Edit inbound rules

Step 27:

Click: Add rule

Step 28:

Type: MySQL/Aurora
Port automatically becomes: 3306

Step 29:

Source:
Select: Anywhere IPv4

Step 30:

Click: Save rules

PART 10 — COPY DATABASE ENDPOINT

Step 31:

Go back to:
RDS

Step 32:

Click database: mydb

Step 33:

Under:
Connectivity & security
Copy:
Endpoint
Example: mydb.abcdefg.us-east-1.rds.amazonaws.com

PART 11 — INSTALL MYSQL WORKBENCH

Step 34:

Open: MySQL Workbench Download Page
Step 35: Download Windows installer.
Step 36: Install normally.

PART 12 — OPEN MYSQL WORKBENCH
Step 37:
Press: Windows Key
Step 38:
Search:
MySQL Workbench
Open it.

PART 13 — CREATE CONNECTION
Step 39:
Click:
+
(Create new connection)
Step 40:
Connection Name: AWS-RDS
Step 41:
Hostname: Paste copied endpoint.
Step 42:
Port: 3306
Step 43:
Username: admin
Step 44:
Click:
Store in Vault
Password:admin123
Step 45:
Click:
Test Connection
Expected:
Successfully connected
Step 46:
Click: OK

PART 14 — OPEN CONNECTION
Step 47:
Double click: AWS-RDS
SQL editor opens.

PART 15 — CREATE TABLE
Step 48:

Run:
```
CREATE DATABASE stud;
USE stud;
CREATE TABLE student(
id INT,
name VARCHAR(50)
);
```
Step 49:
Click:
Execute
(lightning icon)

PART 16 — INSERT DATA
Step 50:
Run:
```
INSERT INTO student VALUES(1,'Rahul');
```
Execute it.

PART 17 — RETRIEVE DATA
Step 51:
Run:
```
SELECT * FROM student;
```
Execute it.
RESULT
Expected output:

id	name
1	Rahul

Q5] Using Amazon Web Services, create and deploy a simple static website using Amazon S3.
Tasks to be Performed

1. Create an S3 bucket for website hosting.
2. Configure the bucket for static website hosting.
3. Upload the following files:
    - index.html (main webpage)
    - At least one image file
4. Apply a bucket policy to enable public access.
5. Set the index document as index.html.
6. Access the website using the generated S3 endpoint URL.
7. Verify that the website loads correctly with all resources.

Mapped to: Experiment 7 but uses bucket policy

Steps:
PART 1 — CREATE WEBSITE FILES
Step 1:

Create a folder on Desktop.
Example:mywebsite

Step 2:

Inside folder create file:
index.html

Step 3:

Right click → Open with Notepad

Step 4:

Paste this code:

```
<!DOCTYPE html>
<html>
<head>
  <title>AWS Static Website</title>
</head>
<body>
<h1>Welcome to My AWS Website</h1>
<p>Website hosted using Amazon S3</p>
<img src="aws.png" width="300">
</body>
</html>
```

Step 5:

Press: Ctrl + S
Save file.

Step 6:

Copy any image into the same folder.
Rename image as: aws.png

Step 7:

Verify folder contains:

- index.html
- aws.png

PART 2 — LOGIN TO AWS

Step 8:

Open AWS Console.
Login using AWS account.

PART 3 — OPEN S3

Step 9:

Search: S3
Open: S3

PART 4 — CREATE BUCKET

Step 10:

Click: Create bucket

Step 11:
Bucket name:
Example: my-static-site-12345
Important:

- Bucket name must be globally unique

If an error comes, add extra numbers.
Step 12:
Region: Keep default.

PART 5 — ENABLE PUBLIC ACCESS
Step 13:
Scroll to: Block Public Access settings
Step 14:
Untick: Block all public access
Step 15:
Tick acknowledgement checkbox.
Step 16:
Keep the remaining settings default.
Step 17:
Click:
Create bucket
Bucket created successfully.

PART 6 — OPEN BUCKET
Step 18:
Click bucket name:
my-static-site-12345

PART 7 — UPLOAD WEBSITE FILES
Step 19:
Click: Upload
Step 20:
Click: Add files
Step 21:
Select:

- index.html
- aws.png

Step 22:
Click:Open
Step 23:
Click:
Upload
Wait until upload completes.

PART 8 — ENABLE STATIC WEBSITE HOSTING

Step 24:
Open:
Properties tab.

Step 25:
Scroll to: Static website hosting

Step 26:
Click: Edit

Step 27:
Select: Enable

Step 28:
Hosting type: Host a static website

Step 29:
Index document: index.html

Step 30:
Click: Save changes

PART 9 — APPLY BUCKET POLICY

Step 31:
Open:
Permissions tab.

Step 32:
Scroll to: Bucket policy

Step 33:
Click: Edit

Step 34:
Paste this policy:

```
{
 "Version":"2012-10-17",
 "Statement":[{
   "Sid":"PublicRead",
   "Effect":"Allow",
   "Principal":"*",
   "Action":"s3:GetObject",
   "Resource":"arn:aws:s3:::my-static-site-12345/*"
 }]
}
```

Step 35:
Replace:
my-static-site-12345
with your actual bucket name.

Step 36:
Click:
Save changes

PART 10 — ACCESS WEBSITE
Step 37:
Go back to:
Properties tab.
Step 38:
Scroll to: Static website hosting
Step 39:
Copy:
Bucket website endpoint
Example:
http://my-static-site-12345.s3-website-us-east-1.amazonaws.com
Step 40:
Open a new browser tab.
Paste endpoint URL.
RESULT
Expected:

- Website opens successfully
- Heading visible
- Image visible correctly

Q6] Using Amazon Web Services, monitor the performance of a virtual server using Amazon EC2 and Amazon CloudWatch.
Tasks to be Performed

1. Launch an EC2 instance using Amazon Linux .
2. Navigate to Amazon CloudWatch from the AWS Management Console.
3. Access Metrics → EC2 to view instance performance data.
4. Monitor key metrics such as:
    - CPUUtilization
    - DiskReadOps
    - NetworkIn/NetworkOut
5. Create a CloudWatch Alarm for the EC2 instance.
6. Configure the alarm condition: trigger when CPU utilization exceeds 70%.
7. Attach a notification action (SNS or email alert).
8. Verify alarm functionality by generating load on the instance.

Mapped to: Lab 6

Steps:

PART 1 — LOGIN TO AWS
Step 1:
Open AWS Console.
Login using AWS account.

PART 2 — LAUNCH EC2 INSTANCE

Step 2:
Search: EC2
Open: EC2
Step 3:
Click: Launch Instance
Step 4:
Instance name: MyServer
Step 5:
AMI:
Select: Amazon Linux 2023
Step 6:
Instance type:t3.micro

Step 7:
Key pair:

- Select existing key pair (can also use the keypair created in 2nd question)

OR

- Create new key pair

Example: mykey
Download .pem file.
Step 8:
Network settings: Keep default.
Step 9:
Click: Launch Instance
Step 10:
Wait until instance state becomes: Running

PART 3 — OPEN CLOUDWATCH
Step 11:
Search: CloudWatch
Open: CloudWatch

PART 4 — VIEW EC2 METRICS
Step 12:
Left menu → Metrics
Step 13:
Click: All metrics
Step 14:
Open: EC2
Step 15:
Click: Per-Instance Metrics

PART 5 — VIEW PERFORMANCE METRICS
Step 16:

Search your instance ID.
Example: i-0123456789abcdef0
Step 17:
View:

- CPUUtilization
- DiskReadOps
- NetworkIn
- NetworkOut

Step 18:
Tick metric checkbox to view graph.

PART 6 — CREATE CLOUDWATCH ALARM

Step 19:
Tick: CPUUtilization
Step 20:
Click:Create alarm

PART 7 — CONFIGURE ALARM CONDITION
Step 21:
Metric:
CPUUtilization
already selected.
Step 22:
Threshold type: Static
Step 23:
Condition: Greater than
Step 24:
Threshold value: 70
This means the alarm triggers when the CPU exceeds 70%.
Step 25:
Click: Next

PART 8 — CREATE SNS NOTIFICATION
Step 26:
Under Notification select: In alarm
Step 27:
Select: Create new topic
Step 28:
Topic name: cpu-alert
Step 29:
Email address: Enter your email.
Example: abc@gmail.com
Step 30:

Click: Create topic
Step 31:
Click: Next

PART 9 — NAME THE ALARM
Step 32:
Alarm name: HighCPUAlarm
Step 33:
Click: Next
Step 34: Review details.
Step 35:
Click: Create alarm
Alarm created successfully.
PART 10 — CONFIRM EMAIL SUBSCRIPTION
Step 36: Open your email inbox.
Step 37: You will receive SNS confirmation mail.
Step 38:
Click: Confirm subscription

PART 11 — CONNECT TO EC2
Windows Users
Step 39:
Open:
PowerShell
Step 40:
Go to the .pem file location.
Example: cd Downloads
Step 41:
Connect to EC2.
Run:
ssh -i mykey.pem ec2-user@public-ip(copy the ip address of the instance)
Example:
ssh -i mykey.pem ec2-user@13.xx.xx.xx
Step 42:
Type: yes
Connected successfully.

PART 12 — GENERATE CPU LOAD
Step 43:
Run this command:
yes > /dev/null &
This continuously increases CPU usage.
Step 44:
Run multiple times for more load.

Example:
yes > /dev/null &
yes > /dev/null &
yes > /dev/null &

PART 13 — VERIFY ALARM
Step 45:
Wait around: 2–5 minutes
Step 46:
Go back to: CloudWatch
Step 47:
Open: Alarms

Step 48:
Check the alarm state.
Expected: In alarm
Step 49:
Check email inbox.
Expected:

- SNS email alert received (click on resubscription tab)

PART 14 — STOP CPU LOAD
Step 50:
Return to the terminal.
Step 51:
Run:
pkill yes
CPU load stops.
RESULT
Expected:

- CPU utilization graph increases
- Alarm state changes to: In alarm
- Email notification received

Q7] Using Amazon Web Services, set up and configure a basic web server using Amazon EC2. You are required to launch a virtual machine, install and run a web server, configure network access, and modify instance resources to meet application requirements.
Steps to Execute:

1. Launch an Amazon EC2 instance using a suitable AMI (e.g., Amazon Linux/Ubuntu).
2. Connect to the instance and install the Apache Web Server.
3. Start and enable the Apache service.
4. Modify the instance's security group to allow HTTP (port 80) access.
5. Verify the web server by accessing it through the instance's public IP address.
6. Resize the instance by:

- Changing the instance type
- Modifying the EBS storage volume

Mapped to: Lab 3

Steps:

PART 1 — LOGIN TO AWS

Step 1:

Open AWS Console.

Login using AWS account.

PART 2 — LAUNCH EC2 INSTANCE

Step 2:

Search: EC2

Open: Amazon EC2

Step 3:

Click: Launch Instance

Step 4:

Instance name: WebServer

Step 5:

AMI:

Select: Amazon Linux 2023

Step 6:

Instance type:t3.micro

Step 7:

Key pair:

Select existing key pair

OR

Create new key pair

Example: mykey

Download .pem file.

Step 8:

Network settings: Keep default.

Step 9:

Click: Launch Instance

Step 10:

Wait until instance state becomes:

Running

PART 3 — CONNECT TO EC2 INSTANCE USING EC2 INSTANCE CONNECT

Step 11:

Select the launched EC2 instance.

Step 12:

Click: Connect

Step 13:

Select tab: EC2 Instance Connect

Step 14:
Click: Connect
Step 15:
Terminal opens successfully.
Expected terminal:
ec2-user@ip-172-31-xx-xx

PART 4 — CONNECT USING SSH (OPTIONAL METHOD)
Windows Users
Step 16:
Open: PowerShell
Step 17:
Go to the .pem file location.
Example: cd Downloads
Step 18:
Connect to EC2 instance.
Run: ssh -i mykey.pem ec2-user@public-ip
Example: ssh -i mykey.pem ec2-user@13.xx.xx.xx
Step 19:
Type: yes
Connected successfully.

PART 5 — INSTALL APACHE WEB SERVER
Step 20:
Update packages.
Run: sudo yum update -y
Step 21:
Install Apache Web Server.
Run: sudo yum install httpd -y

PART 6 — START AND ENABLE APACHE SERVICE
Step 22:
Start Apache service.
Run: sudo systemctl start httpd
Step 23:
Enable Apache service.
Run: sudo systemctl enable httpd
Step 24:
Check Apache status.
Run: sudo systemctl status httpd
Expected: active (running)

PART 7 — ALLOW HTTP (PORT 80) ACCESS
Step 25: Go back to the EC2 Dashboard.

Step 26: Select the EC2 instance.
Step 27: Open: Security tab
Step 28: Click attached: Security Group
Step 29:
Click: Edit Inbound Rules
Step 30:
Click: Add Rule
Step 31:
Configure:

Type	Protocol	Port Range	Source
HTTP	TCP	80	Anywhere

Step 32:
Click: Save Rules

PART 8 — VERIFY WEB SERVER
Step 33:
Copy Public IPv4 Address of the instance.
Example: 13.xx.xx.xx
Step 34: Open browser.
Step 35:
Enter: http://public-ip
Example: http://13.xx.xx.xx
Step 36:
Expected: Apache default web page displayed successfully.

PART 9 — CREATE SAMPLE WEB PAGE
Step 37:
Return to EC2 terminal.
Step 38:
Go to the Apache web directory.
Run: cd /var/www/html
Step 39:
Create an HTML file.
Run: sudo nano index.html
Step 40:
Add:
<h1>Welcome to AWS EC2 Web Server</h1>
Step 41:
Save file.
Step 42: Refresh browser.
Expected: Custom webpage displayed.

PART 10 — CHANGE INSTANCE TYPE

Step 43: Go to EC2 Dashboard.
Step 44: Select the instance.
Step 45:
Click:
Instance State → Stop Instance
Step 46:
Wait until instance state becomes: Stopped
Step 47:
Click:
Actions → Instance Settings → Change Instance Type
Step 48:
Select new instance type.
Example: t2.small
Step 49: Click: Apply
Step 50: Start the instance again.

PART 11 — MODIFY EBS STORAGE VOLUME
Step 51:
Go to:
EC2 Dashboard → Volumes
Step 52:
Select attached EBS volume.
Step 53:
Click:
Actions → Modify Volume
Step 54:
Increase storage size.
Example: 8 GB → 16 GB
Step 55: Click: Modify
RESULT
Expected:

- EC2 instance launched successfully
- Apache Web Server installed and running
- HTTP Port 80 enabled
- Web server accessible through Public IP
- Custom webpage created successfully
- Instance type changed successfully
- EBS storage volume modified successfully

Q8] Design and configure a Virtual Private Cloud (VPC) with public and private subnets across multiple Availability Zones, configure routing and security settings, and launch an EC2 instance as a web server accessible through the internet.
Task
Create a custom VPC with public and private subnets.

Configure Internet Gateway, NAT Gateway, and route tables.
Create additional subnets in another Availability Zone.
Configure subnet associations and routing.
Create a security group to allow HTTP access.
Launch an EC2 instance inside the public subnet.
Configure the EC2 instance as a web server using user data scripts.
Verify web server accessibility using the public DNS of the instance.

Mapped to: Lab 2

Steps:

PART 1 — LOGIN TO AWS

Step 1: Open AWS Console.
Login using AWS account.

PART 2 — CREATE CUSTOM VPC

Step 2:
Search: VPC
Open: Amazon Virtual Private Cloud

Step 3:
Left menu → Your VPCs

Step 4:
Click: Create VPC

Step 5:
Resources to create: VPC only

Step 6:
VPC Name: MyVPC

Step 7:
IPv4 CIDR Block: 10.0.0.0/16

Step 8:
Click: Create VPC

PART 3 — CREATE PUBLIC SUBNET

Step 9:
Left menu → Subnets

Step 10:
Click: Create subnet

Step 11:
Select VPC: MyVPC

Step 12:
Subnet name:Public-Subnet-1

Step 13:
Availability Zone:
Example: ap-south-1a

Step 14:
IPv4 CIDR block subnet: 10.0.1.0/24

Step 15:
Click: Create subnet

PART 4 — CREATE PRIVATE SUBNET
Step 16:
Click: Create subnet
Step 17:
Select VPC: MyVPC
Step 18:
Subnet name: Private-Subnet-1
Step 19:
Availability Zone: ap-south-1a
Step 20:
IPv4 CIDR block subnet: 10.0.2.0/24
Step 21:
Click: Create subnet

PART 5 — CREATE SUBNETS IN ANOTHER AVAILABILITY ZONE
Step 22:
Create another public subnet.
Subnet name: Public-Subnet-2
Availability Zone: ap-south-1b
CIDR: 10.0.3.0/24
Step 23:
Create another private subnet.
Subnet name: Private-Subnet-2
Availability Zone: ap-south-1b
CIDR: 10.0.4.0/24

PART 6 — CREATE INTERNET GATEWAY
Step 24:
Left menu → Internet Gateways
Step 25:
Click: Create internet gateway
Step 26:
Name: MyIGW
Step 27:
Click: Create internet gateway
Step 28:
Click: Attach to VPC
Step 29:
Select: MyVPC
Step 30:
Click: Attach internet gateway

PART 7 — CREATE PUBLIC ROUTE TABLE

Step 31:
Left menu → Route Tables

Step 32:
Click: Create route table

Step 33:
Name: Public-RT

Step 34:
Select VPC: MyVPC

Step 35:
Click: Create route table

PART 8 — ADD INTERNET ROUTE

Step 36:
Select: Public-RT

Step 37:
Open: Routes tab

Step 38:
Click: Edit routes

Step 39:
Click: Add route

Step 40:
Destination: 0.0.0.0/0

Step 41:
Target: Select Internet Gateway

Step 42:
Choose: MyIGW

Step 43:
Click: Save changes

PART 9 — ASSOCIATE PUBLIC SUBNETS

Step 44:
Open: Subnet Associations tab

Step 45:
Click: Edit subnet associations

Step 46:
Select:

- Public-Subnet-1
- Public-Subnet-2

Step 47:
Click: Save associations

PART 10 — CREATE ELASTIC IP FOR NAT GATEWAY

Step 48:
Left menu → Elastic IPs
Step 49:
Click: Allocate Elastic IP address
Step 50:
Click: Allocate

PART 11 — CREATE NAT GATEWAY
Step 51:
Left menu → NAT Gateways
Step 52:
Click: Create NAT Gateway
Step 53:
Name: MyNAT
Availability: select zonal
Step 54:
Subnet:
Select: Public-Subnet-1
Step 55:
Elastic IP:
Select allocated Elastic IP
Step 56:
Click: Create NAT Gateway
Wait until status becomes:
Available

PART 12 — CREATE PRIVATE ROUTE TABLE
Step 57:
Create a route table.
Name: Private-RT
Step 58:
Select VPC: MyVPC
Step 59:
Click: Create route table

PART 13 — ADD NAT ROUTE
Step 60:
Select: Private-RT
Step 61:
Routes → Edit routes
Step 62:
Add route:
Destination: 0.0.0.0/0
Target: Select NAT Gateway

Step 63:
Choose: MyNAT
Step 64:
Click: Save changes

PART 14 — ASSOCIATE PRIVATE SUBNETS
Step 65:
Subnet Associations → Edit subnet associations
Step 66:
Select:

- Private-Subnet-1
- Private-Subnet-2

Step 67:
Click: Save associations

PART 15 — CREATE SECURITY GROUP
Step 68:
Go to EC2 Dashboard.
Step 69:
Left menu → Security Groups
Step 70:
Click: Create security group
Step 71:
Security Group Name: WebSG
Step 72:
Select VPC: MyVPC
Step 73:
Inbound Rules → Add Rule
Configure:

Type	Protocol	Port	Source
HTTP	TCP	80	Anywhere
SSH	TCP	22	Anywhere

Step 74:
Click: Create security group

PART 16 — LAUNCH EC2 INSTANCE
Step 75: Go to EC2 Dashboard.
Step 76:
Click: Launch Instance
Step 77:
Instance Name: WebServer

Step 78:
AMI: Amazon Linux 2023
Step 79:
Instance Type:t3.micro
Step 80:
Key Pair: Select existing key pair
Step 81:
Network Settings:

- Select VPC: MyVPC
- Select Subnet: Public-Subnet-1
- Auto-assign Public IP: Enable
- Security Group: WebSG

PART 17 — CONFIGURE USER DATA SCRIPT
Step 82:
Open: Advanced Details
Step 83:
Under User Data paste:

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
echo "<h1>Welcome to AWS Web Server</h1>" > /var/www/html/index.html
```

Step 84:
Click: Launch Instance
Step 85:
Wait until instance state becomes:
Running

PART 18 — VERIFY WEB SERVER
Step 86: Select the EC2 instance.
Step 87:
Copy Public ip adress.
Example: ec2-13-xx-xx-xx.ap-south-1.compute.amazonaws.com
Step 88: Open browser.
Step 89:
Enter: http://public-ip
Step 90:
Expected Output: Welcome to AWS Web Server

Q9]Problem: Set up AWS CloudTrail for Monitoring API Calls
Tasks to be Performed
1. Open CloudTrail from Console.

2. Create a Trail & Apply to all regions.
3. Create a new S3 bucket for log storage.
4. Perform an AWS action (e.g., stop/start EC2).
5. Check if CloudTrail logs the action.
Steps:
PART 1 — LOGIN TO AWS
Step 1:
Open AWS Console.
Login using AWS account.

PART 2 — OPEN CLOUDTRAIL
Step 2:
Search:CloudTrail
Open: CloudTrail

PART 3 — CREATE TRAIL
Step 3:
Left menu → Trails
Step 4:
Click: Create trail

PART 4 — CONFIGURE TRAIL
Step 5:
Trail name: MyTrail
Step 6:
Under:
Choose log events
Keep default: Management events
Step 7:
Under:
Storage location
Select: Create new S3 bucket
Step 8:
S3 bucket name:
Example: cloudtrail-logs-12345
Important:

- Bucket name must be globally unique

If an error comes, add extra numbers.
Step 9:
Uncheck Log file SSE-KMS encryption
Keep default settings.

PART 5 — APPLY TO ALL REGIONS
Step 10:

Under:
Apply trail to all regions
Select: Yes
Step 11:
Keep all remaining settings default.
Step 12:
Click:
Create trail
Trail created successfully.

PART 6 — PERFORM AWS ACTION
Example: Stop and Start EC2 Instance
Step 13:
Search:EC2
Open: EC2
Step 14:
Left menu → Instances
Step 15:
Select any running EC2 instance.
Example: MyServer
Step 16:
Click: Instance state
Step 17:
Click:Stop instance
Step 18:
Confirm: Stop
Wait until the instance stops.
Step 19:
Again click: Instance state
Step 20:
Click: Start instance
Instance starts again.

PART 7 — VERIFY CLOUDTRAIL LOGS
Step 21:
Go back to: CloudTrail
Step 22:
Left menu → Event history
Step 23:
You will see logged AWS actions.
Examples:

- StopInstances
- StartInstances

Step 24:

Click event name: StopInstances

Step 25:

View details:

- Username
- Event time
- Source IP
- Event name
- AWS service

PART 8 — VERIFY LOG FILES IN S3

Step 26:

Search:S3

Open: S3

Step 27:

Open created bucket:

Example: cloudtrail-logs-12345

Step 28:

Open folders inside the bucket.

You will see CloudTrail log files stored automatically.

Q10] Using Amazon Web Services and Terraform, implement automated cloud infrastructure provisioning through Infrastructure as Code (IaC).

Tasks to be Performed

1. Install and configure Terraform on your system.
2. Create a working directory and initialize Terraform using terraform init.
3. Configure AWS provider with appropriate credentials.
4. Write a Terraform configuration file to:
5. Launch an Amazon EC2 instance
6. Define instance type and AMI
7. Configure security group to allow SSH/HTTP access
8. Use Terraform commands to preview infrastructure changes and provision resources
9. Verify that the EC2 instance is successfully created on AWS.
10. Access the instance using SSH and validate deployment.
11. Terminate all created resources.

Steps:

PART 1 — INSTALL TERRAFORM

Step 1:

Open: Terraform Download Page

Step 2:

Download:Windows AMD64

(for Windows systems)

Step 3:

Extract downloaded ZIP file.

Step 4:

Copy: terraform.exe
Step 5:
Create folder: C:\Terraform
Paste: terraform.exe
inside it.

PART 2 — ADD TERRAFORM TO PATH
Step 6:
Search:Environment Variables
Open:Edit the system environment variables
Step 7:
Click: Environment Variables
Step 8:
Under: System Variables
Select: Path
Click: Edit
Step 9:
Click:
New
Add: C:\Terraform
Step 10:
Click: OK
for all windows.

PART 3 — VERIFY TERRAFORM INSTALLATION
Step 11:
Open: PowerShell
Step 12:
Run:terraform -version
Expected: Terraform v1.x.x

PART 4 — INSTALL AWS CLI
Step 13:
Open: AWS CLI Download Page
Step 14: Download and install AWS CLI.
Step 15:
Verify installation.
Run: aws --version

PART 5 — CREATE IAM USER FOR TERRAFORM
Step 16:
Open AWS Console.

Step 17:

Search:IAM
Open: IAM
Step 18:
Left menu → Users
Step 19:
Click: Create user
Step 20:
Username: terraform-user
Step 21:
Click: Next
Step 22:
Select: Attach policies directly
Step 23:
Search: AdministratorAccess
Tick checkbox.
Step 24:
Click: Next
Step 25:
Click: Create user

PART 6 — CREATE ACCESS KEY
Step 26:
Open created user:
terraform-user
Step 27:
Open: Security credentials
Step 28:
Scroll to: Access keys
Step 29:
Click: Create access key
Step 30:
Select: Command Line Interface (CLI)
Step 31: Tick acknowledgement checkbox.
Step 32:
Click: Next
Step 33:
Click:Create access key
Step 34:
Copy:

- Access Key ID
- Secret Access Key

Important: Save them safely.

PART 7 — CONFIGURE AWS CLI

Step 35:
Open: PowerShell
Step 36:
Run: aws configure
Step 37:
Enter:

- Access Key ID
- Secret Access Key
- Region

Example: us-east-1
Step 38:
Output format: json

PART 8 — CREATE TERRAFORM PROJECT
Step 39:
Create folder on Desktop:
terraform-project
Step 40:
Open folder in:
VS Code
or Notepad.

PART 9 — CREATE TERRAFORM FILE
Step 41:
Create file: main.tf
Step 42:
Paste this code:

```
provider "aws" {
 region = "us-east-1"
}

resource "aws_security_group" "web_sg" {
 name = "web-sg"

 ingress {
   from_port   = 22
   to_port     = 22
   protocol    = "tcp"
   cidr_blocks = ["0.0.0.0/0"]
 }

 ingress {
   from_port   = 80
   to_port     = 80
```

```
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
 }
 egress {
  from_port   = 0
  to_port     = 0
  protocol    = "-1"
  cidr_blocks = ["0.0.0.0/0"]
 }
}
resource "aws_instance" "myserver" {
 ami           = "ami-0c02fb55956c7d316"
 instance_type = "t2.micro"
 security_groups = [aws_security_group.web_sg.name]
 tags = {
  Name = "TerraformServer"
 }
}
```

Step 43: Save file.

PART 10 — INITIALIZE TERRAFORM

Step 44:

Open PowerShell inside the project folder.

Step 45:

Run: terraform init

Terraform initialization starts.

Expected: Terraform has been successfully initialized

PART 11 — PREVIEW INFRASTRUCTURE

Step 46:

Run: terraform plan

This previews infrastructure changes.

PART 12 — CREATE INFRASTRUCTURE

Step 47:

Run: terraform apply

Step 48:

Type: yes

Press Enter.

Step 49:

Terraform creates:

- EC2 instance
- Security group

Expected: Apply complete

PART 13 — VERIFY EC2 INSTANCE
Step 50:
Open AWS Console.
Step 51:
Search: EC2
Open: EC2
Step 52:
Open: Instances
Step 53:
Verify instance: TerraformServer is running.

PART 14 — CONNECT USING SSH
Windows Users
Step 54:
Open: PowerShell
Step 55:
Go to .pem file location.
Example:
cd Downloads
Step 56:
Copy Public IP from EC2.
Step 57:
Connect: ssh -i mykey.pem ec2-user@public-ip
Example: ssh -i mykey.pem ec2-user@13.xx.xx.xx
Step 58:
Type: yes
Connected successfully.

PART 15 — VALIDATE DEPLOYMENT
Step 59:
Run: hostname
Expected: ip-xx-xx-xx-xx
Step 60:
Run: pwd
Linux environment working successfully.

PART 16 — TERMINATE RESOURCES
Step 61:
Return to the PowerShell project folder.
Step 62:
Run: terraform destroy

Step 63:

Type: yes
Press Enter.
Step 64:
Terraform deletes:

- EC2 instance
- Security group

Expected: Destroy complete

Q11] Using Amazon Web Services, implement a Platform as a Service (PaaS) solution by deploying a web application using AWS Elastic Beanstalk.
Tasks to be Performed

1. Open AWS Elastic Beanstalk from the AWS Management Console.
2. Create a new Elastic Beanstalk application.
3. Configure an environment (Web Server Environment).
4. Select a suitable platform (e.g., Node.js, Python, or PHP).
5. Upload application source code or use a sample application.
6. Configure environment settings:
    - Instance type (within Free Tier)
    - Key pair for access
    - Auto-scaling configuration
7. Deploy the application
8. Access the application using the generated URL.
9. Monitor application health and performance using the dashboard.
10. Manage the environment (restart, rebuild, or terminate).

Mapped to: Activity

Steps:
PART 1 — LOGIN TO AWS
Step 1:
Open AWS Console.
Login using AWS account.

PART 2 — OPEN ELASTIC BEANSTALK
Step 2:
Search: Elastic Beanstalk
Open: Elastic Beanstalk

PART 3 — CREATE APPLICATION
Step 3:
Click:
Create application

PART 4 — CONFIGURE APPLICATION
Step 4:
Application name:MyWebApp

Step 5:
Environment name:
Keep auto-generated names.
Example: MyWebApp-env
Step 6:
Domain:
Keep default autogenerated domain.

PART 5 — SELECT PLATFORM
Step 7:
Platform:
Select: Node.js
You may also select:

- Python
- PHP

Step 8:
Platform branch:
Keep default.
Step 9:
Platform version:
Keep the recommended version.

PART 6 — APPLICATION CODE
Step 10:
Under:
Application code
Select:
Sample application

PART 7 — PRESETS
Step 11:
Under:
Presets
Select:
Single instance
This keeps deployment Free Tier friendly.

PART 8 — SERVICE ACCESS
Step 12:
Expand:
Service access
Step 13:
Service role:
Keep default.

Example: aws-elasticbeanstalk-service-role
Step 14:
EC2 key pair:
Select your key pair.
Example: mykey

PART 9 — NETWORKING SETTINGS
Step 15:
Keep default VPC settings.
Step 16:
Public IP address:
Keep: Activated

PART 10 — INSTANCE SETTINGS
Step 17:
Expand: Instance traffic and scaling
Step 18:
Instance type: t3.micro
Step 19:
Environment type: Load Balanced
Auto scaling:
Keep minimum:
1
and maximum:
1

PART 11 — MONITORING SETTINGS
Step 20:
Monitoring:
Keep default settings.

PART 12 — CREATE APPLICATION
Step 21:
Scroll down.
Click:
Submit
or
Create application

PART 13 — DEPLOYMENT PROCESS
Step 22:
Elastic Beanstalk starts creating:

- EC2 instance
- Security group

- Environment
- Application deployment

Step 23:
Wait: 5–10 minutes
Step 24:
Environment health becomes:
OK
and status becomes:
Ready

PART 14 — ACCESS APPLICATION
Step 25:
Locate generated application URL.
Example: http://mywebapp-env.eba-xxxxx.us-east-1.elasticbeanstalk.com
Step 26:
Click the URL.
RESULT
Expected:
Sample application webpage opens successfully.

PART 15 — MONITOR APPLICATION
Step 27:
Inside dashboard monitor:

- Health
- Requests
- CPU usage
- Instances

Step 28:
Verify health status:
OK

PART 16 — RESTART APPLICATION
Step 29:
Click: Actions
Step 30:
Click: Restart app server(s)
Step 31:
Confirm restart.

PART 17 — REBUILD ENVIRONMENT
Step 32:
Click: Actions
Step 33:
Click: Rebuild environment

Step 34:
Confirm rebuild.

PART 18 — TERMINATE ENVIRONMENT
Step 35:
Click: Actions
Step 36:
Click: Terminate environment
Step 37:
Type environment name.
Example: MyWebApp-env
Step 38:
Click: Terminate